Quiz App

Table of Contents

Introduction

Our project is an educational web application called *Quiz App* for taking, creating and sharing quizzes, which can be used in a school setting or to learn on your own.

Target users

The primary users of the quiz web application are students and teachers in educational or training environments. The platforms are designed to support digital learning by making it easy to create, take and manage quizzes. For students the application provides engaging way to test their knowledge, receive instant feedback and track their progress. Teachers can create and customize quizzes, monitor results and analyse student performance.

In addition to its educational use, the application also appeals to general users who enjoy quizzes for fun or competition. Anyone with an interest in quizzes can register themselves and afterwards both take and create their own quizzes to challenge friends or family. This adds a social and entertaining element to the platform, making it suitable for both formal and daily use.

Overall, the application targets a broad audience both for learning and enjoyment. By combining education and entertainment, the quiz system promotes both learning ang engagement and in an accessible and interactive way.

Tech stack

The version of Node.js is v22.18.0
The version of ASP.NET is 8.0.20
The version of React is 19.2.0
The version of TypeScript is ~5.8.3

Running the project

To run the project make sure you have Node.js and npm installed. They can be downloaded from this website: https://nodejs.org/en

For running the full project, navigate to the QuizApp folder, and run

npm install
npm run build
npm run preview

to run the project as intended.

If *npm run build* returns the error

‘tsc’ is not recognized as an internal or external” command

you will need to run *npm install typescript* first to successfully build and run the project.

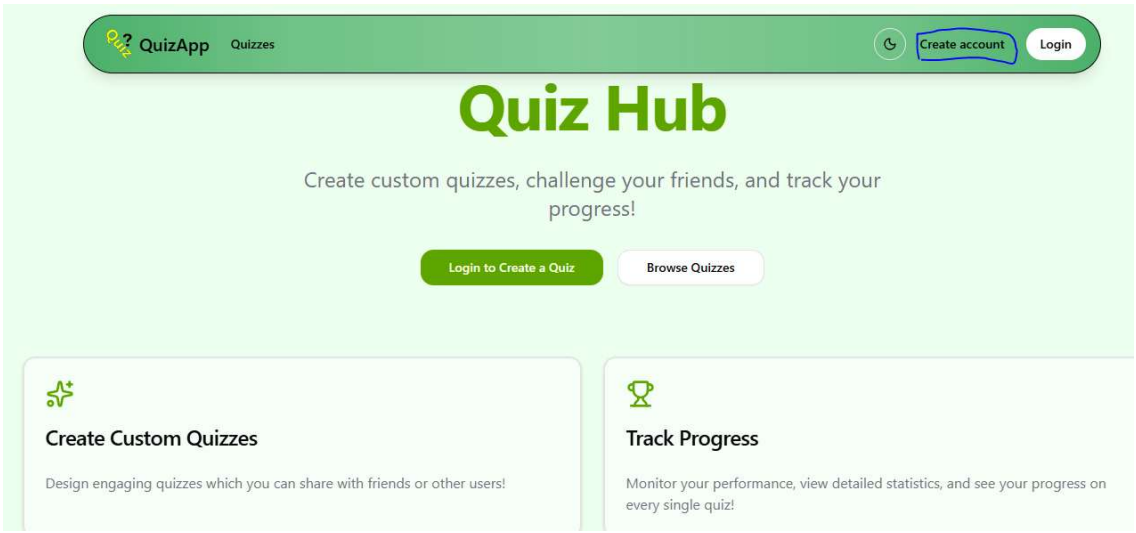
If errors still occur, navigate to the api folder and run

dotnet restore
dotnet ef database update

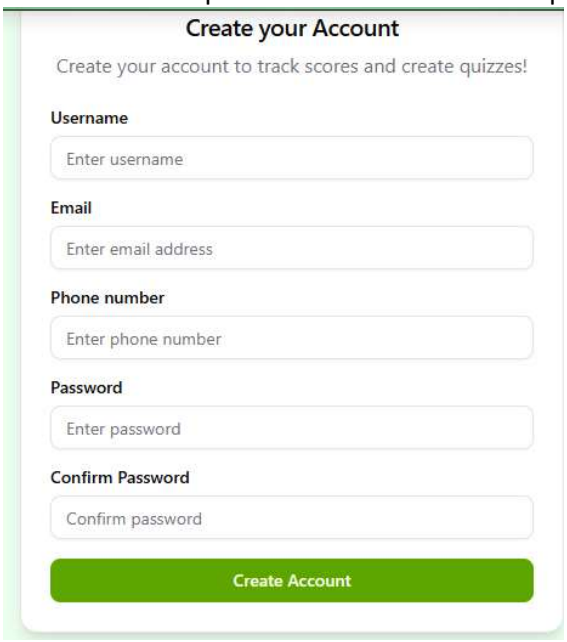
And go back to QuizApp folder and run *npm run build* and *npm run preview.*

User creation

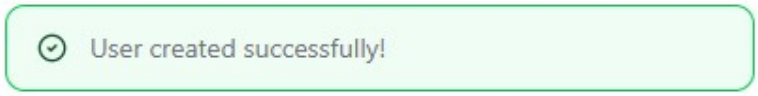
1. Click on “Create account



2. Fill out the required information in the empty fields

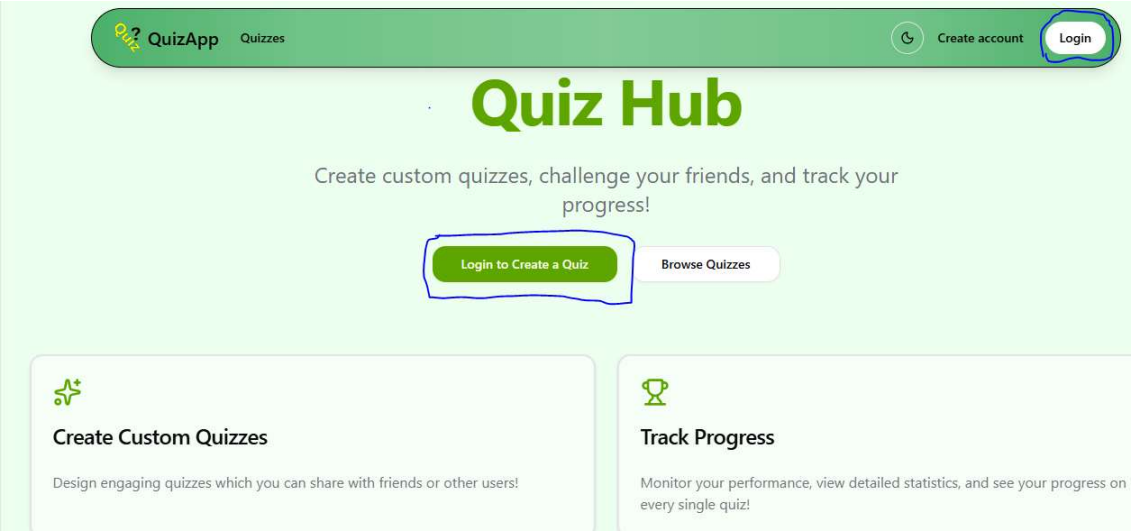


3. If no errors occur during account creation, you will get a success message:

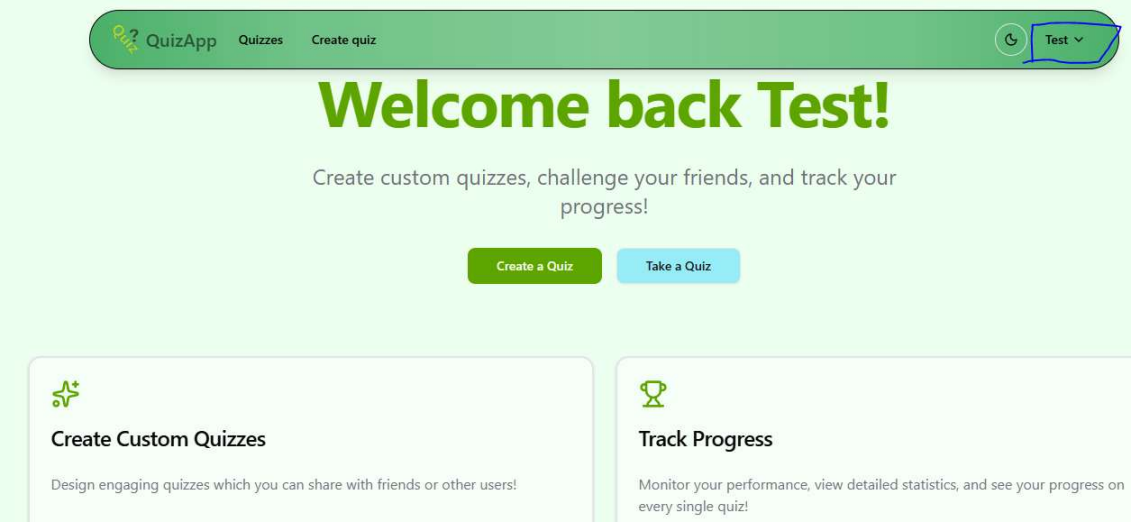


User log in process

1. Click on “Login to Create a Quiz” in the middle of the page or “Login” in the top right side of the navbar



2. If the login is successful your username will appear in the top right corner. In this example, we have logged into an account called “Test”. You will also see “Welcome back” and your username under the navbar.



Logging in as administrator

There is a standard Admin user on the website, with the username is ‘admin’ and the password is ‘Admin1234’. The password is easy on purpose to make it easier to login into the admin account. If the web application we made would have gone live, we would have

chosen a much harder and complex password to make the ‘admin’ user secure. Also, we would remove any hardcoded account creations, by seeding in the admin account on initial database creation, before removing any trace of the seeding.

AI usage during development

We used generative AI for conversion from one framework to another, namely Bootstrap to Tailwind CSS, using shadcn/ui components.

The project was originally written using react-bootstrap, v. 2.10.10, based on Bootstrap 5.3, to create the general structure and functionality of the web design. We wanted to change from Bootstrap due to wanting a more modern design, which is easier to troubleshoot and change down the road in future implementations. We also saw that the project design-wise suffered from the classic Bootstrap look, with a lot of recognizable elements from other Bootstrap applications online.

Using Claude, namely the model Sonnet 4.5, we ended up converting the project component by component and page by page to implement Tailwind CSS instead. Since Tailwind CSS is a utility framework instead of a component framework like Bootstrap, we also ended up using the shadcn/ui, as this was recommended by Claude, and easy to implement by adding only specific components when these were needed, reducing the size of the application.

Conversion started by requesting first that the navbar should be converted over, by providing Claude with the full source code for the component. The conversion was not very accurate for all pages, and as such, manual changes, not using AI, to the HTML layout, had to be done for the page to be functional. The main failures of AI were mainly contrast issues between foreground text and background, as well as translating margins and padding between elements, due to the custom navbar. Also, due to the already implemented Bootstrap styling and needing to keep this as a reference while converting, Claude often forgot or even removed the ‘!’-tag from multiple CSS tags, which was needed to force the styling from both shadcn/ui and Tailwind CSS.

AI was also used for implementing specifically the user role function to control user access, also done by Claude. Changes had to be done in the JWT token generation for authorizing access on the frontend, while database seeding had to be created to accommodate for the created roles. All of this was generated by AI, including internal logging for database seeding and migration, due to access functionality being faulty after creation of roles, which was due to an error in the creation of the JWT token, which had to be fixed manually.

The repository pattern was also created using AI. During development, we ended up mixing business logic, backend requests and responses often together in what ended up being quite large and confusing files. Using Claude, we extracted and placed major parts of the source code into either new or existing files to create a structure based on separation. This was also done in the backend, by supplying AI with a template repository pattern, as well as the source code from the now massive backend controllers which were mixed with both requests to the database as well as business logic.

Finally, as the base project had a horrible file structure which was difficult to navigate, with source files being in wrong places, we also used generative AI as a suggestion to how we should structure the code, requesting suggestions for where specific files should be placed, in specific folders.

In tasks not necessarily directly related to code, generative AI, specifically ChatGPT and Claude, were both used in gaining knowledge of .NET, React, Vite, Bootstrap and Tailwind CSS. Especially in cases where both the documentation and the extended course reading were either not specific or confusing, generative AI was asked to provide a simpler explanation and/or examples for usage of requested functions. Furthermore, in the case of errors, AI was also at times used to check for errors in the code, either by supplying the code and requesting a check for errors, or by writing logging lines for the group to use for debugging the application and finding the error.

Contributions

Everyone did contribute and help each other when developing the web application in all areas of development, but our main tasks are listed below.

Student 1:
Coordinate coding and merging all code together, coordinate writing, writing of introduction and architecture, merging all writing together.

Student 2:
Writing and coding of data access, writing and coding of business logic and controllers.

Student 3:
Writing and coding of error handling, logging, and input validation, in close collaboration with others, and writing and coding of authentication.

Student 4:
Writing and coding of front-end and back-end communication (AJAX calls), and user interface design, user experience design, user test, and writing.

Requirements Analysis

The following sections outlines the requirements for the application.

Value proposition

People need our quiz web application because it makes creating and taking quizzes in a fast, fun and engaging way. Whether for entertainment, team building or learning, our quiz application turns knowledge into a social and interactive experience.

Required functionalities

There are many required functionalities in our quiz web application.

1. User accounts and profiles
   People must have the possibility of making users in the application so they can make their own quizzes, edit them and delete them. They also need to see their scores and their profile where they will be able to edit their email, phone number and password. People can also take quizzes without making a user or logging in but then their points won't be stored.

2. User database
   We need a database that contains the user information after someone makes a user account.

3. Quiz creation page
   Logged in users must be able to access the quiz creation page. This page will have the functionalities to make quizzes. The page must contain input field for Quiz Title and Quiz Description. It must also have input field for the question, number of points, answer options and selection of the correct answer. The page must also have the possibility to add and delete questions and answer options.

4. Quiz Database
   There must be a quiz database that stores the quizzes that authorised users have made.

5. Quizzes page
   The quizzes page will display all available quizzes fetched from the quiz database. Each quiz will have a start button there will also be a delete button visible for the admin user. The delete button is there for the admin user to make it possible for the admin user to delete quizzes in an easy way.

6. Quiz taking page
   After starting the quiz users will be sent to the Quiz taking page where they answer the questions. They will see the questions then they chose an answer option. After that they will be able to go to the next question. They can also go back to the previous question. Lastly, they can click the "submit quiz" button and then they get their score.

7. Dark mode function
   We will implement a button so users can switch between light and dark mode.

8. Page for seeing scores for logged in users
   In this page logged in users will be able to see the scores they got on different quizzes; they will also be able to see what questions they got right and wrong.

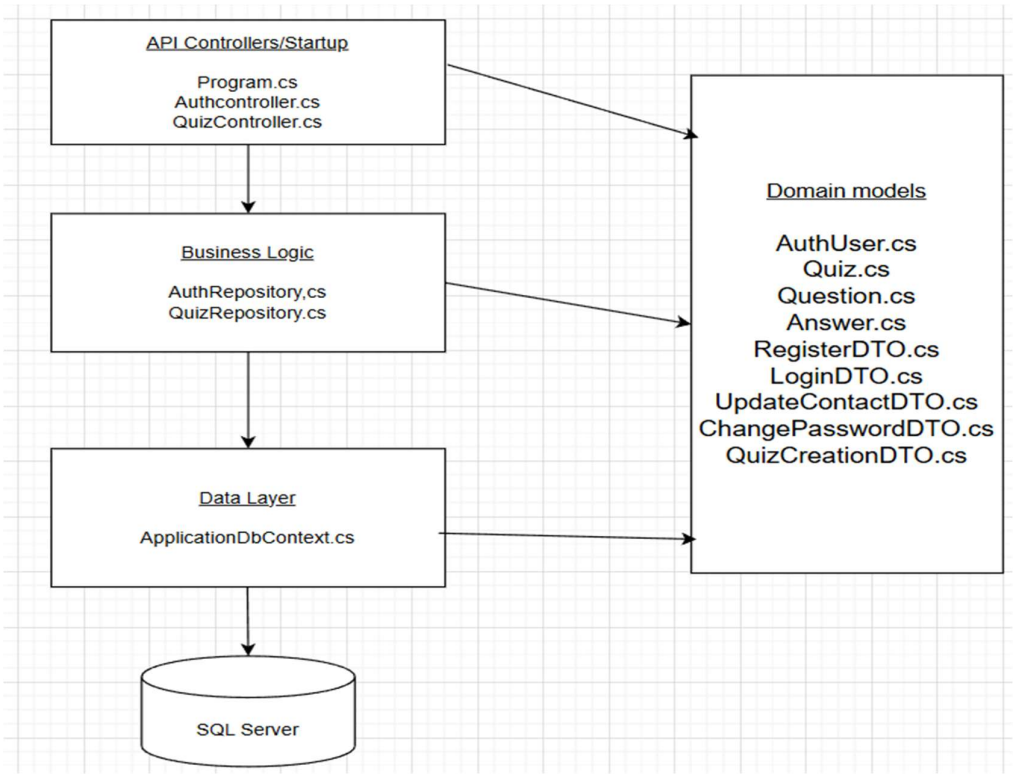
9. My quizzes page
   In this page users will be able to see their own quizzes, they will also be able to update their quizzes and delete them if they want to.

10. Profile page
    In the profile page users will be able to see their profile, here they will see their username, role, email, phone number they will also be able to change their Email address, phone number and password. Lastly, they will also be able to delete their account if they want to.

11. Admin control page
    The admin control page will only be displayed for the admin user, here the admin user will be able to delete users, it will also be a search function so the admin user can find users to delete in a fast and easy way.

12. Input validation
    It's important that the web application has input validation both in the frontend and the backend.

Software Architecture



The application uses a layered architecture to keep the system clean and maintainable.

- API controllers/Startup
  (Program.cs, AuthController.cs, QuizController.cs)
  Handle HTTP requests and responses. They only coordinate actions and forward work to the Business Logic Layer.

- Business Logic
  (AuthRepository.cs, QuizRepository.cs)
  Contains the core rules of the application, such as authentication, quiz handling, and validations. Keeps controllers simple.

- Data Access Layer
  (ApplicationDbContext.cs)
  Manages all communication with the SQL database using Entity Framework Core.

- Domain Models
  (AuthUser.cs, Quiz.cs, Question.cs, DTOs, etc.)
  Define the shared data structures used between all layers

- SQL Server
  Stores all persistent data. The DAL translates models in SQL tables

---

Functionality

The main functionality is the create and take quizzes. Sub-functionalities are expressed in the table below.

Main Functionality	Sub-Functionalities
Create and take quizzes	• Create quizzes • Take quizzes (logged in + guests) • Submit and view scores
User accounts & profiles	• Register users • Login • Edit profile • View score page • View/edit/delete "My quizzes"
Quiz creation	• Add/remove questions • Add/remove answer options
Quiz taking	• Navigate questions • Submit quizzes • Guest scores not stored
Admin features	• Delete any quiz • Access admin page/user control • Delete users
Supporting features	• Frontend/backed input validation • Light/dark mode • Scores page
Databases	• User database • Quiz database • Required for storing quizzes, answers, user info, scores

Data models

Model	Purpose	Key Properties	Why It Is Needed
Quiz Model	Represents a quiz created by a user; contains all information required for quiz functionality.	• Id • Title (max 200 chars) • Description • UserId (creator) • Navigation to User • Questions collection	• Store and organize quizzes in DB • Connect each quiz to its creator • Store related questions for accurate quiz taking & scoring
Question Model	Stores individual questions and links them to the quiz.	• QuestionId • Question text • Points • QuizId (FK) • Navigation to Quiz • ICollection Answers	• Store questions in DB • Link questions to the correct quiz
Answer Model	Represents answer options and identifies the correct answer.	• Id • AnswerText • IsCorrect flag • QuestionId (FK) • Navigation to Question	• Store answer options & correct answer • Link answers to the correct question
Quiz Submission Model	Represents a user's quiz submission and their scoring results.	• Id • UserId • QuizId • Score • MaxScore • QuestionSnapshot • AnswerSnapshot • DateTime	• Store quiz submissions & results • Allow users to view scores later in score history

Auth User Model	Represents authenticated users; handles identity and security.	*Inherits IdentityUser:* • Id • Username • Email • PasswordHash	• Secure user management • Registration and login • Role-based access • Reference for quizzes and user-specific data

Frontend modules and their functionality

Page	Purpose / Functionality
UserControl	Allows admin users to delete users from the system.
HomePage	The main landing page of the application.
AllQuizzes	Displays all available quizzes; admin can delete any quizzes here.
MyQuizzes	Allows users to view, update, and delete their own created quizzes.
QuizCreatePage	Page where users create new quizzes.
QuizTakingPage	Page where logged-in users and guests take quizzes.
QuizUpdatePage	Page where users edit/update their existing quizzes.
MyScores	Shows users their past quiz scores and details on correct/incorrect answers.
UserCreationPage	Page for creating a new user account.
LoginPage	Page where users log in to their account.
UserProfile	Shows user details (username, role, email, phone). Users can update email, phone number, password, and delete their account.
NotFound	Simple page showing the user that they are on a non-existent page, when a 404 error is returned.

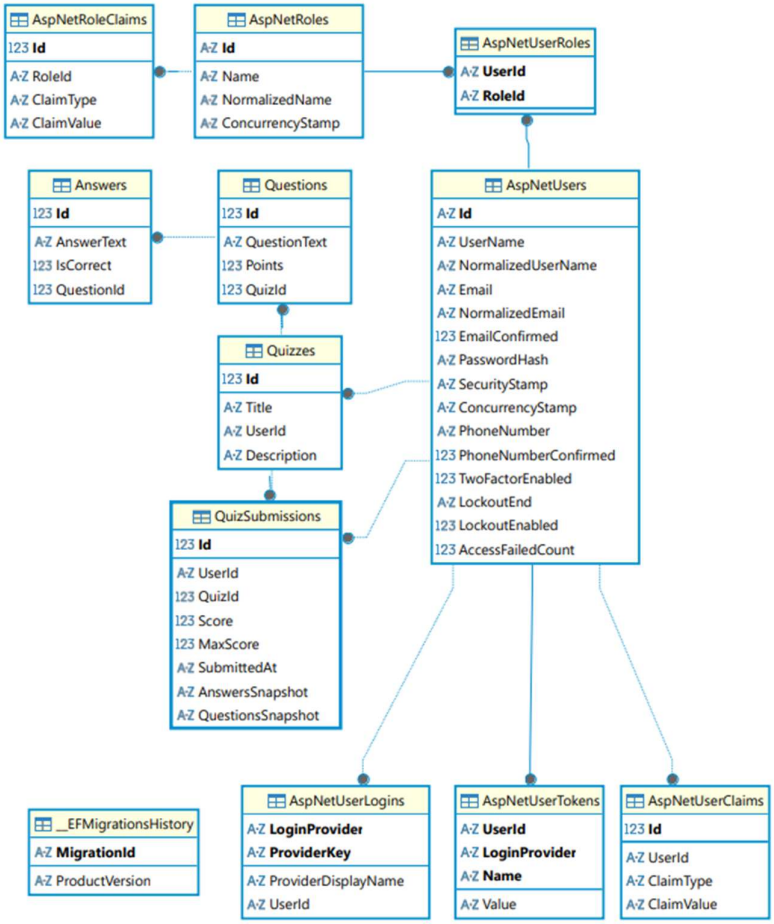
All these pages are required to make the web application work in a good and effective way.

Data Access

We use a relational database with Entity Framework Core (EF Core) the reason why we choose this is because:

- The quiz data fits naturally into tables (users, quizzes, questions, answers).
- EF Core makes it easy to create, update and query the database without writing SQL
- ASP.NET identity works directly with EF Core, so handling users and roles becomes much simpler
- It is reliable, fast, and easy to set up for both local development and deployment

ER-diagram



This entity relation diagram shows the structure of our database. It was generated using the open-source database management tool DBeaver.

For general development usage

All AspNet-tables stores user information for authentication and identity management, while custom tables Quizzes, QuizSubmissions, Questions and Answers store information about quizzes, linked by using the Id-field from the AspNetUsers table.

Expanded explanation

All ASP.Net tables (Prefix AspNet: Users, UserRoles, Roles) are created using ASP.Net Core Identity, which is constructed in the AuthUserDbContext.cs file. However, by using the Core Identity class IdentityDbContext, extra tables not used are also constructed due to the base constructor, these tables being AspNetUserTokens, AspNetUserClaims, AspNetUserLogins and AspNetRoleClaims.

In the database, AspNetUsers is the main table, responsible for assigning a user ID, and keeping track of password hashes and other field usually found in a user profile, with its contents being of each individual user registered in the application. The user ID is what connects the user with their individual roles, which are also constructed using the Core Identity model.

The user ID from the AspNetUsers is also linked with the custom tables Quizzes and QuizSubmissions. The Quizzes-table is the main table, where there is a 1-to-many connection to both QuizSubmissions and Questions, while having a 1-to-1 connection with AspNetUsers, keeping track of the user ID owning the quiz, who can perform editing and removing said quiz.

The Questions-table contains a question ID as its main identifier and what quiz ID the question is linked with, while containing information about the question text, total points given, and how many points said question gives. Meanwhile, the separate answers are kept in a separate data table, with their own ID, connected to their questions ID, keeping information about their text and if the answer is correct or not, by using a boolean value, 0 or 1. All questions and answers connected to a quiz are deleted if the quiz is deleted.

Every QuizSubmissions entry is linked by connecting each separate submission using the quiz ID retrieved from the Quizzes-table, while connecting it with a user performing the quiz, and is the table we retrieve information from when displaying a user's score on each individual quiz. Every entry, while not having an explicit connection to either one of Questions or Answers, contains a snapshot of the questions and correct answers of each quiz submitted at the time of submission. By avoiding the explicit links, the submission uses its own information about questions and answers, which lets the user check their quizzes even if the quiz answers and questions would be edited or even deleted. However, if a quiz is deleted, all submissions will be deleted as well, in case the quiz is either malicious or unwanted, as we do not want to keep information on the database which could recreate the quiz in question.

As for the tables not used in the database structure: AspNetUserTokens could in production be used for resetting passwords, changing emails and other short-lived use-cases, using local concurrency tokens using short expiration dates/times. AspNetUserLogins would be used if OpenID Connect is used, as it stores the third-party login provider and the associated key. The claims-tables would be used if the application uses a claim-based authentication flow, which is not needed in this case, but could be implemented if the system required more roles with more defined access control. Finally, the _EFMigrationsHistory does not have any function other than being a logger for migration changes when the database is changed in the code.

Database connection

ASP.NET Core connects to the database using a class called ApplicationDbContext, which inherits from DbContext.

In Program.cs, we register the context like this:

```
builder.Services.AddDbContext<AuthUserDbContext>(options =>
{
    options.UseSqlite(builder.Configuration["ConnectionStrings:AuthUserDbContextConnection"]);
});
```

This tells ASP.NET Core:

- Which database provider to use (SQLite/SQL Server)
- Which connection string to connect with.

The DbContext is then automatically injected into repositories and used to read/write data.

Repository pattern

We use a repository for each main area example from the project:

- IAuthRepository
- IQuizRepository

```
20 references
private readonly IAuthRepository _authRepository;
```

```
private readonly IQuizRepository _quizRepository;
```

Each repository handles all the data access for that specific part of the app.

Why we need it:

- Keeps the controllers clean (controllers do not talk directly to the database)
- Makes the backend easier to maintain

- Makes unit testing possible (We can mock repositories instead of using a real database)
- Follows clean architecture and good programming practice

How is the API service layer designed (frontend)? Why do we need it?
The API service layer is designed to POST JSON data from the frontend to the server and GET data from the server and into the frontend. It's also for PUT to update quizzes and users. We have AdminService, AuthService and QuizService.

Unit Tests

The test project has a dedicated environment, enforcing the separation of concerns principle. More reasons for this include that the requirement differs from the main project, keeping it simple and well organized, and they don't need to be included if/when the main project is deployed.

The project is created by using the command *"dotnet new xunit -o api.Tests" k*eeping within the scope of the DotNet framework. The test framework itself is xUnit, and uses the Moq Library and follows an AAA pattern. AAA stands for Arrange, Act, and Assert for a clear structure that all the test follows.

Implemented tests

For the quiz entity, a series of tests was implemented, using different tests, including both negative and positive, but also testing Authorization/validation. These tests cover all the CRUD aspects for the entity, from creating a quiz and adding it to the database, updating the quiz, getting both individual quizzes based on ID, and the full list of all quizzes and deleting quizzes.

The AuthControllertest verifies all critical authentication and profile-related functionality in the backend. It includes both positive and negative test cases for user registration, login, profile retrieval, updating contact information and password changes. Using mocked repositories and an in-memory JWT configuration, the tests isolate the controller logic and therefore can function without a real database. Authenticated and unauthenticated scenarios are simulated by injecting custom user claims to test the logic in the application. The tests confirm that valid requests return the expected result while invalid input or missing authentication correctly produces error responses and messages.

Unit testing is needed for multiple purposes. In this project they are used to ensure confirm that the api handles errors in a graceful and appropriate way. It also helps in verifying that we have proper authorization checks and business logic. Lastly it serves as a double check list, when refactoring or making changes in the source code.

Expected results of tests

The results of the tests is a response with a corresponding HTTP status.
Here is a table of expected results, created by Microsoft Copilot:

Test	Expected Result	HTTP Status
GetQuizzes_ok	Returns OkObjectResult with quiz list	200
GetQuizzes_NoQuizzes	Returns OkObjectResult with empty list	200
GetQuizzes_Exception	Returns ObjectResult	500
GetQuiz_ok	Returns OkObjectResult with quiz data	200
GetQuiz_WhenIt_DoesNotExist	Returns NotFoundObjectResult	404
GetQuiz_Exception	Returns ObjectResult	500
CreateQuiz_ok	Returns OkObjectResult with created quiz	200
CreateQuiz_Unauthorized	Returns UnauthorizedObjectResult	401
CreateQuiz_BadRequest	Returns BadRequestObjectResult	400
CreateQuiz_Error	Returns ObjectResult	500
UpdateQuiz_ok	Returns OkObjectResult	200
UpdateQuiz_NoUser	Returns UnauthorizedObjectResult	401
UpdateQuiz_BadData	Returns BadRequestObjectResult	400
DeleteQuiz_Successfull	Returns NoContentResult	204
DeleteQuiz_Error	Returns ObjectResult	500

Static code analysis

We sent our web application through the static code analysis tool/web page Snyk, checking what types of vulnerabilities which we might have missed during development of the project. Through this tool, we updated multiple frameworks and libraries containing known vulnerabilities, as well as changes to the code to avoid both XSS (cross-site-scripting) and SQL injection. Currently, the only vulnerabilities that Snyk reports are regarding the hardcoded credentials in the code for creating mock users and the main admin user. These are credentials that should only be used for testing purposes and is something that would be removed if the project is pushed to production.

User Experience

The user interface is designed to be clean and intuitive with a focus on easy navigation and simplicity. The navigation bar is at the top of the page and contains navigation the different pages in an easy way. Everything is designed in a way to make it easy for users to read the texts and make navigation between pages easy and satisfying.

User test

We did conduct user testing, we asked some students if they could navigate through our quiz application, we asked them about if they could navigate through the pages in an easy way. We also asked them if they liked the design and if there was something they would have changed. They tested both in a normal webpage view and in the phone view so we could get feedback on both. Some feedback we got was that we had a bit too dark colors in the start, the buttons was also a bit small, and some pages was a bit hard to navigate to and understand. The feedback from the user tests did help us a lot to improve our solution. We changed to bigger buttons, lighter colours with the correct contrast. We also made the pages easier to navigate to and understand.

Usability

Our web application got good usability, the navbar always always keeps the same position to not create confusion. The pages also follow the same color themes, so users don't get confused or annoyed. The buttons and texts are also big to make navigation easy, in the navbar the text the user have the mouse over gets grayed out a bit so it's easier for the user to see what button is hovered. Everything is designed to make the usability the best way possible.

Accessibility

The UI is design with accessibility in mind. The use of strong contrast, appropriate element names for people using screen readers and clear navigation.

Conclusion and Reflection

In this project, we have gained insight into full-stack development and working as a team. We have learned how to use the DotNet framework to build a web application. This includes using tools, libraries and other frameworks like Razor pages, cshtml, React, Bootstrap and Tailwind. For most of us, it was also the first-time programming in C#. Combining and learning all these new tools have been the most challenging part of the project, other challenged we faced was to integrate code written by individual members together. To improve we could maybe have tried to use extreme programming or other agile methods in our process. We had weekly meetings throughout the semester, but most of the programming was done individual with the task we delegated at our meetings.

As a group we managed to create an environment where all ideas to either improve or change the functionality or the user interface were welcomed and discussed during our weekly meetings. The ideas were then implemented by the individual who had the idea to begin with and showed to the others at the next meeting. Everybody had inputs as to how to application should function and two members had the primary vision for GUI. Using both our creative and functional skills we managed to create and easy to use, visually pleasing and fully functional webapp for creating, taking and sharing quizzes.

Everybody in the group valued the processes of this project, as it has given us some experience and insight into what it means to be a full stack developer. A job positions most of the group could see themself in their professional carrier. It also allowed us the easier start and work on personal projects for our portfolio, which is an element we have heard employers value when hiring for junior positions.